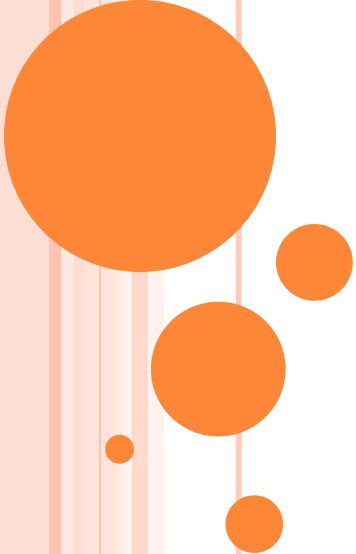




MATLAB PARTE 3

Array mono e bidimensionali:

Assegnazione, operazioni su array, funzioni che operano su array

ARRAY IN MATLAB

- Non occorre dichiarare le dimensioni né il tipo degli array (dichiarazione implicita e dimensionamento automatico)
- gli indici delle componenti partono sempre e solo da 1
- gli operatori algebrici e le funzioni sono già definiti per array

ARRAY MONODIMENSIONALI

```
>> x=[ 10.5 -2 0]; y= [2,3,5,7]% vettori riga  
>> z= [2;3;5;-3.4]           % vettore colonna  
>>w=[-4  
      6  
      8]                     % vettore colonna  
>> x(4)=6.5
```

- Un array si può assegnare usando parentesi quadre;
- Ogni componente dell'array si ottiene col nome del vettore seguito dall'indice della sua posizione in parentesi tonde

COSTRUZIONE DI ARRAY CON ELEMENTI EQUISPAZIATI

- Specificando l'incremento

a:h:b $\longleftrightarrow$ [a , a+h , a+2h , , b]

se h=1 si può omettere

- Specificando il numero di punti

linspace(a,b,n)

genera n punti equispaziati tra a e b. Se n=100 si può omettere

Esempio

```
>> x=linspace(-1,1,5)
```

```
x =
```

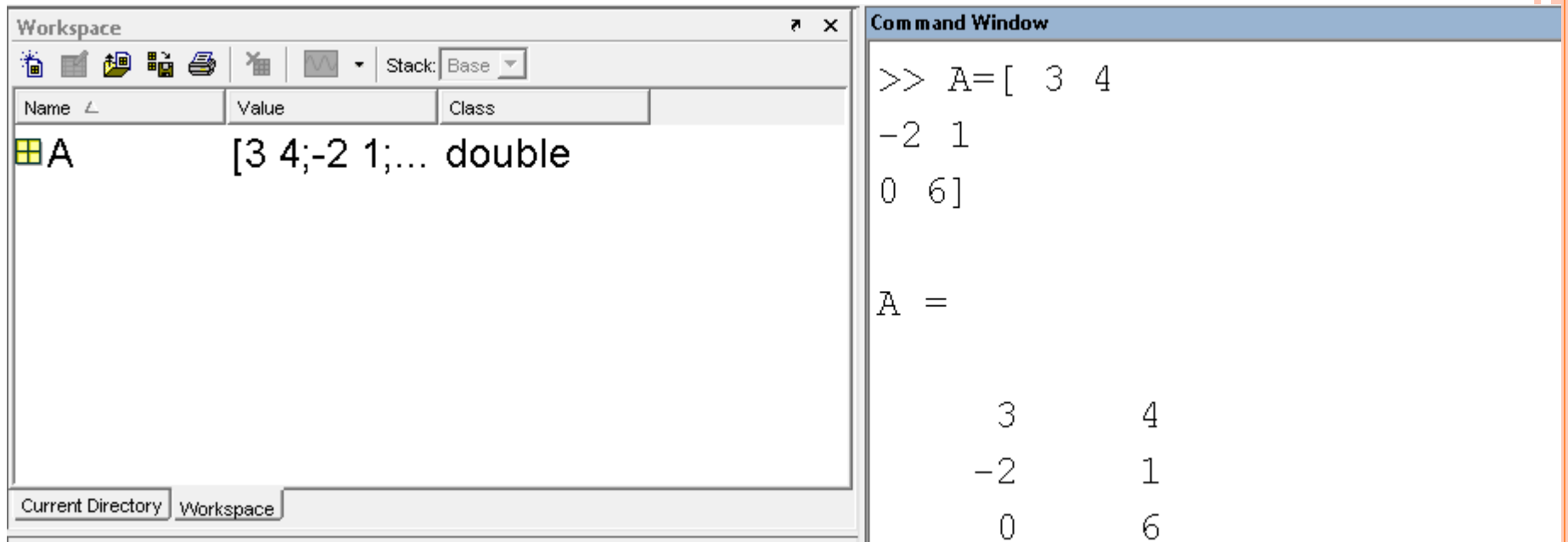
```
-1.0000 -0.5000 0 0.5000 1.0000
```

ARRAY BIDIMENSIONALI

```
>> A=[ 1 -2.4  5  
      8  -1  2.3]  
>> B=[ 3.5 6; 4 -3.2; 4 -2]  
>> C=[ 3,-9; 2,1;0 -2]  
>> A(1,3)= -3.5
```

- Gli elementi di ogni riga sono separati da spazi o virgole;
- Le righe sono separate da punti e virgole o 'a capo'
- ogni elemento si ottiene col nome della matrice vettore seguito dagli indici di riga e di colonna in parentesi tonde

ARRAY EDITOR



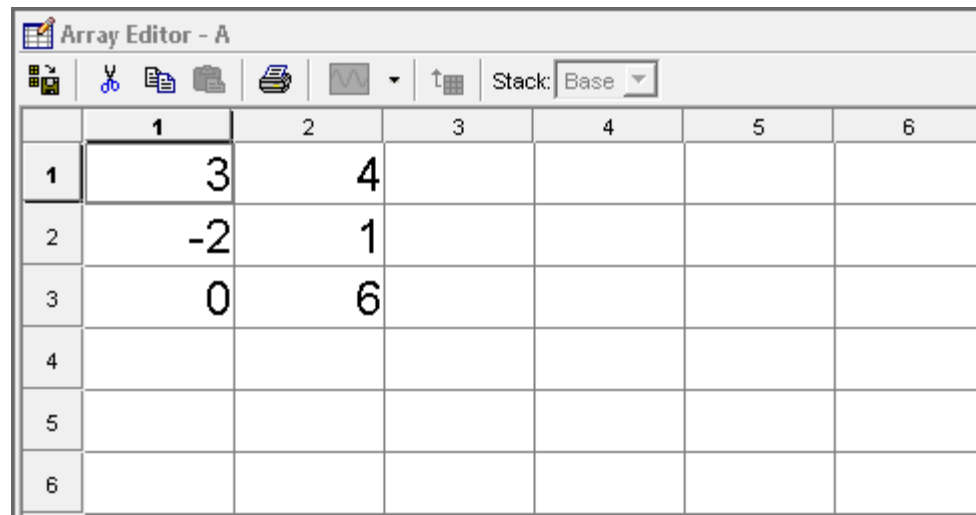
The screenshot shows the MATLAB interface. The **Workspace** window on the left displays a table with the following data:

Name	Value	Class
A	[3 4;-2 1;...]	double

The **Command Window** on the right shows the command `>> A=[ 3 4` followed by a new line `-2 1` and another new line `0 6]`. Below the command, the output is displayed as:

```
A =  
  
     3     4  
    -2     1  
     0     6
```

Cliccando 2 volte sulla variabile A nel workspace si avvia l'array editor:



The screenshot shows the **Array Editor - A** window. It contains a table with 6 columns and 6 rows. The first three rows contain the values of the array A, and the remaining three rows are empty.

	1	2	3	4	5	6
1	3	4				
2	-2	1				
3	0	6				
4						
5						
6						

FUNZIONI SIZE E LENGTH

- length restituisce la lunghezza di un vettore
- size restituisce le dimensioni del vettore o della matrice

```
>> x=[-4.2 5.1 6]; A=[ 1 -2.4 5; 8 -1 2.3];  
>> length(x)  
ans =  
      3  
>> size (A)  
ans =  
      2      3
```

SOTTOMATRICI

- A è una matrice
- x e y sono vettori
- $A(x,y)$ = Matrice che si ottiene utilizzando gli elementi di x come indici di riga e quelli di y come indici di colonna

ESEMPI

```
>> A = [ 2  3  4  5  6  
        -4 -5 -6 -7 -8  
         3  5  7  9  1  
         4  6  8 10 12  
        -2 -3 -4 -5 -6];
```

```
>> v = [1  3  5];
```

```
>> b = A(v, 2);
```

b contiene
 $A(1,2)$, $A(3,2)$, $A(5,2)$

```
>> e = A(1:2, 4:5)
```

e contiene la sottomatrice di **A**
data dall'intersezione delle
righe 1 e 2 con le colonne 4 e 5.

```
>> f = A(1:2, :)
```

f contiene le prime 2 righe di **A**

```
>> d = zeros(3);
```

```
>> d(:,1) = A(v, 2);
```

la prima colonna di **d** viene
sostituita con $A(1,2)$, $A(3,2)$, $A(5,2)$

NOTAZIONE COLON :

- La notazione *colon* serve ad accedere a sottomatrici
- Minimizza l'uso dei loop

Esempi

- $A(:,3)$ è la terza colonna di A
- $A(1:4,:)$ è la sottomatrice di A contenente le prime 4 righe di A
- $A(1:4, 3)$ è il vettore contenente i primi 4 elementi della terza colonna di A
- $A(:, [2\ 4])$ è la sottomatrice contenente la II e la IV colonna di A
- $A(:, [2\ 4\ 5]) = B(:, 1:3)$ sostituisce le colonne di indice 2, 4 e 5 di A con le colonne di indice 1, 2 e 3 di B
- $x = x(n:-1:1)$ inverte l'ordine degli elementi di x

ESERCIZIO

```
<< A = [ 1  5  8; 84  81  7; 12  34  71];
```

```
<< A(1,1)
```

```
<< A(3,3)
```

```
<< A(5,2)
```

```
<< A(1:2, :)
```

```
<< A(:, 1)
```

```
<< A(3, :)
```

```
<< A(:, 2:3)
```

```
<< A(:, [1 3])
```

```
<< A(:, 3:4)
```

**Qual è l'output
di questi comandi?**

Operazioni su array

Matlab esegue automaticamente le operazioni algebriche sugli array

+	Somma
-	Differenza
*	Prodotto righe per colonne
^	Elevazione a potenza
'	Trasposizione
\	Divisione a sinistra ($A^{-1}*B$)
/	Divisione a destra ($A*B^{-1}$)

La operazioni sono consentite solo se le matrici hanno dimensioni compatibili!

Operazioni su matrici

'	Trasposta
inv	Inversa
det	Determinante
eig	Autovalori ed autovettori
rank	rango

ESEMPI

```
>> A=[3 -1 4; -7 0 6];  
B=[ -2 0 1; 3 11 1];  
C=[ 5 1 -7; -3 2 8; 1 -1 8];
```

```
>> A+B
```

```
ans =
```

```
    1    -1     5  
   -4    11     7
```

```
>> A*B
```

```
??? Error using ==> mtimes
```

```
Inner matrix dimensions must agree.
```

```
>> A*C
```

```
ans =
```

```
    22    -3     3  
   -29   -13    97
```

OPERAZIONI PUNTO

- Le operazioni punto agiscono su array delle stesse dimensioni
- $.$ * prodotto elemento per elemento
- $.$ / divisione elemento per elemento
- $.$ ^ elevamento a potenza elemento per elemento

ESEMPI

```
>> x=[-2 3 0 7];  
>> y=[-4 2 11 3];  
>> x.*y  
ans =  
     8     6     0    21  
D=[1 1; 1 1];  
>> D^2  
ans =  
     2     2  
     2     2  
>> D.^2  
ans =  
     1     1  
     1     1
```


- Si possono applicare funzioni matematiche a vettori e matrici; esse agiscono sui singoli elementi dei vettori e delle matrici

```
>> z=[3 -6 1.2 -7];  
>> abs(z)  
ans =  
    3.0000    6.0000    1.2000    7.0000
```

Esercizio

1. Calcolare le radici quadrate dei primi 10 numeri interi. (sugg. usare la funzione **sqrt**)
2. Calcolare i quadrati di 10 numeri equispaziati tra -1 e 1.

- Fra due matrici di eguali dimensioni si possono utilizzare gli operatori relazionali

$<$	minore
$<=$	minore o uguale
$>$	maggiore
$>=$	maggiore o uguale
$==$	uguale
$\neq$	diverso

Il confronto avviene elemento per elemento ed il risultato è una matrice i cui elementi sono solo 0 (falso) e 1 (vero)

OPERATORI LOGICI SU MATRICI

& (AND), | (OR), ~ (NOT)

L'applicazione a matrici avviene elemento per elemento.

Sia p un vettore:

$$\text{any}(p) = \begin{cases} 1 & \text{se almeno un elemento di } p \neq 0 \\ 0 & \text{se tutti gli elementi di } p \text{ sono } = 0 \end{cases}$$

$$\text{all}(p) = \begin{cases} 1 & \text{se tutti gli elementi di } p \text{ sono } \neq 0 \\ 0 & \text{se almeno un elemento di } p \text{ è } = 0 \end{cases}$$

$\text{find}(p)$ = vettore contenente gli indici di elementi di $p \neq 0$

ESEGUI

```
<< p = [5    2   -1   23  -10  -45    0   12];
```

```
<< a = max(p);
```

```
<< b = max(abs(p))
```

```
<< help max
```

```
<< [c, i] =max(abs(p));
```

(Ad es. in eliminazione di Gauss con pivoting)

```
[amax, ipiv] = max(abs(A(k:n, k)))
```

Esegui le stesse istruzioni sulla matrice

```
>> p=[-9  0 -3 ; 1 6 -3; 0 -2 5; 2 4 -8]
```

FUNZIONI PER LA COSTRUZIONE DI MATRICI

- Consultare l'help in linea per la sintassi:
 - `zeros(n,m)` Matrice $n \times m$ con tutti elementi = 0
 - `ones(n,m)` Matrice $n \times m$ con tutti elementi = 1
 - `eye(n,m)` Matrice $n \times m$ con 1 sulla diagonale e 0 sugli elementi extradiagonali
 - `diag` Per costruire matrici diagonali, o estrarre la diagonale delle matrici
 - `rand` per costruire matrici con numeri casuali
- Per matrici quadrate basta indicare una sola dimensione: esempio `zeros(n)`, `ones(n)` ...

FUNZIONI PER LA MANIPOLAZIONE DI MATRICI

- W sia un vettore; allora:
- $\text{diag}(W)$ è una matrice con elementi di W sulla diagonale e 0 altrove
- W sia una matrice; allora:
- $\text{diag}(W)$ è il vettore contenente la diagonale di W
- $\text{tril}(W,k)$ estrae il triangolo inferiore di W partendo dalla diagonale k -ma
- $\text{triu}(W,k)$ estrae il triangolo superiore di W partendo dalla diagonale k -ma.
- Se k è omissso, si intende la diagonale principale

ALTRE FUNZIONI OPERANTI SU MATRICI

- Le funzioni operano sulle singole colonne della matrice W
- $\text{max}(W)$ calcola il massimo fra gli elementi
- $\text{min}(W)$ calcola il minimo fra gli elementi
- $\text{sum}(W)$ calcola la somma degli elementi
- $\text{prod}(W)$ calcola il prodotto degli elementi
- $\text{sort}(W)$ ordina gli elementi in ordine crescente
- $\text{mean}(W)$ calcola la media degli elementi
- $[y,i]=\text{max}(W)$ restituisce la massima componente(y) e la sua posizione